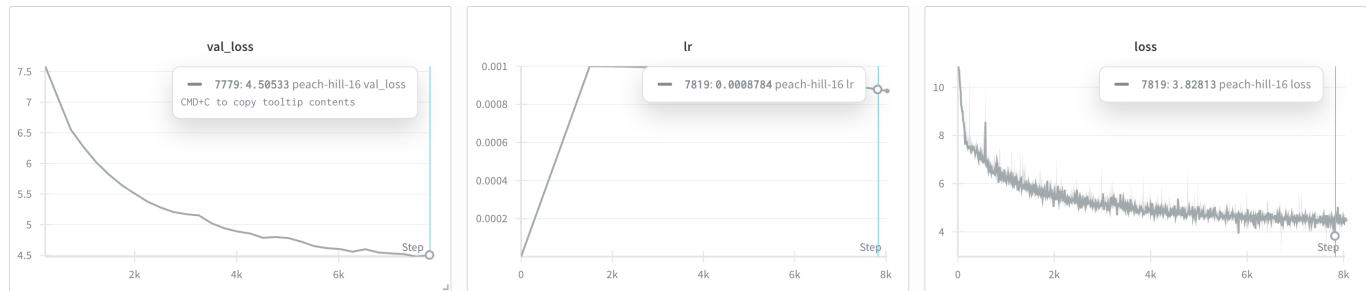


Feb. 8th, 2026

Experiment: OWT

Finished an 1.5 compute-hr run on a RTX Pro 6000 pod with OWT. The resultant model `owt_b16_wtie_100m_32b_1024ctx` achieves a validation loss of ~4.5:



The generated text isn't perfect but is somewhat sensical:

Sangeetha was in the park. The location of the famous 86-year-old throng and clearly shouldn't be available at all on a freight train and train. And cruising all weekend again to distant villages on North Central Coast, then west west of New York, the line between North Central and Lebine with the same height as the Greatfield line that fell from New York along-decker to the Deepwater valley.

If you had a serious lift in the road that may have to be fine, this is the first of three days, and yet the road was covered to an electronic town of almost 100,000 people: the Greatfield intersection of Edinne County and Amdown, Surrey County, on the South end of Manhattan. In 2009, the Grand Central and Lower East Side posted a human cost of nearly \$2.2 million in winning bids from poor urbanites. An estimated \$500 million in profit was the result of \$100 million in the construction and maintenance of Manhattan's café, Macquarie last year.

But then again, the report included a detailed analysis of the design and design factors that made and timing most relevant, along with the usual pattern of bars on the inside of buildings.

The resultant model is undertrained, trained on a 500M tokens out of a 2B token corpus. An additional 1.5-3.0 hours of training will yield much better results.

This model encompasses the following optimizations:

- Torch [compile](#) is used to improve throughput.
- BF16 replaces FP32 to improve compute throughput and reduce memory usage; tensor cores do not handle FP32, so using high-precision floating point leaves a lot of perf on the table.
- Context window is set to 1024 to encompass more sophisticated texts.
- Attention layers are bumped from 4 to 6 to increase model capacity.
- `d_model` is bumped from 512 to 1024, also increasing model capacity.
- Weight tying is used to reduce memory footprint.

Feb. 7th, 2026

GPU Choice / Economics

I noticed the RTX Pro 6000 is another option I've been looking at. I might consider it for my OWT training run.

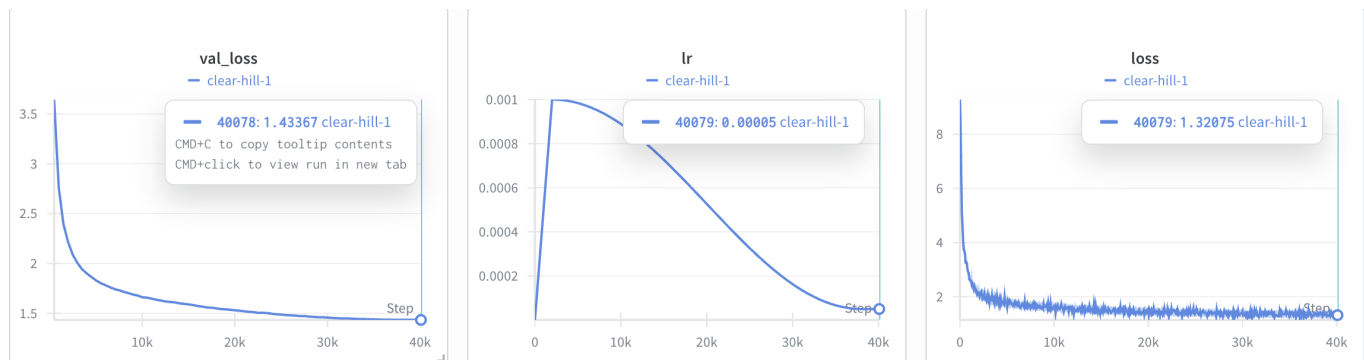
Experiment: TinyStories Final

8:52 AM

Tuning the learning rate a bit more. Since the model is small, I've increased the initial learning rate
 $0.0005 \rightarrow 0.001$.

9:34 PM

Achieved sub-1.45 validation loss! The trick was to pick a higher starting LR, which seems to be fairly stable to smaller models. This was achieved by tiny_stories_1hi5lo.



Experiment: OWT

9:58 PM

Some rough planning for a serious training rough. I'm allocating around ~0.5-1.5 hours of RTX PRO 6000 GPU time for serious runs; some parameter tuning will continue to use RTX 4090 to reduce costs.

The OWT training set contains 2,727,120,452 tokens, yielding a ~100M parameter model based on Chinchilla scaling laws; performance optimality >> inference cost in this scenario.

There are a couple of sub-experiments I'd like to try out:

1. **Default.** Keep the same config as TSv2. This model will be undertrained but will serve as a baseline, catching any potential errors (eg. tokenization).
2. **50M.** Basic 50M parameter run on an RTX 4090, and some other tricks!
 - Weight tying will be used to reduce parameter counts, used in GPT-2 and other models. This should save around 50% of params due to vocab_size being fairly large.
 - Switch from fp32 to bf16.
 - Increase `d_model` from 512 $\rightarrow$ 768, matching smaller GPT-2 models.
 - Increase batch size from 32 $\rightarrow$ 64, improving vRAM usage.
 - Increase context length from 256 $\rightarrow$ 1024. This is a big jump but is necessary since OWT text samples are much longer than TSv2.
 - Increase `d_ff` from 1344 $\rightarrow$ 2016 to satisfy $\frac{8}{3}d_{\text{model}}$ constraint.
 - Maintain 30k iterations to ensure ~2B tokens usage during training.

Feb. 6th, 2026

Continuing the final training run for TinyStoriesV2.

Experiment: TinyStories Final

Attempted with `tiny_stories_final` but wasn't able to hit 1.45 val loss requirement; stuck at ~1.50 val loss. Will increase total steps to 40k to match expectations.

I'm using RTX 4090 instances, which handle ~15 steps/s at \$0.60/hr. This is roughly a doubling in perf compared to A40s with ~50% price premium; however, VRAM does take a cut at 24GB v. 48GB.

1:06 PM

With a more aggressive LR scheduler loss is still stuck at 1.53. Trying with a different LR scheduler config.



3:11 PM

Previous run hit 1.47, trying again with a slightly higher learning rate. Attempted to decrease beta values but had mixed results:



Jan. 21st, 2026

Running a final training run for TinyStoriesV2. Results should be coming soon.

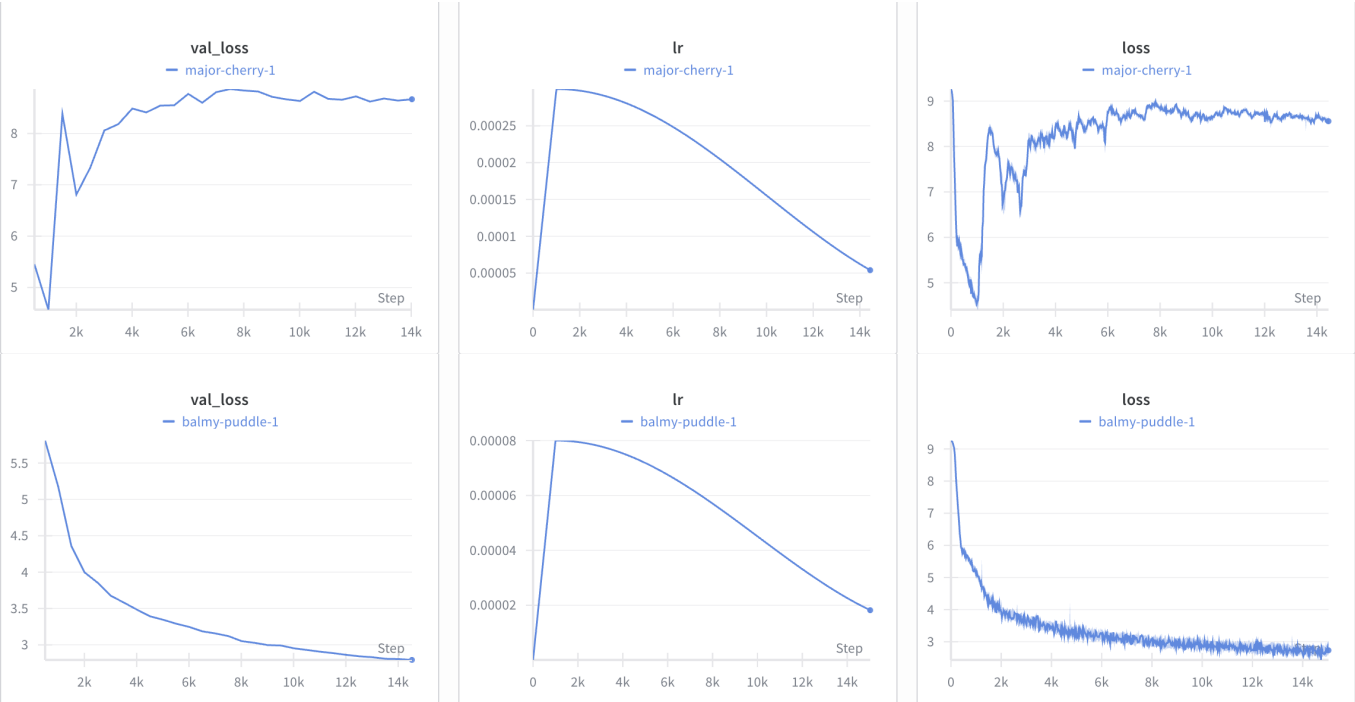
Jan. 20th, 2026

Today was spent finishing up some remaining TinyStories experiments.

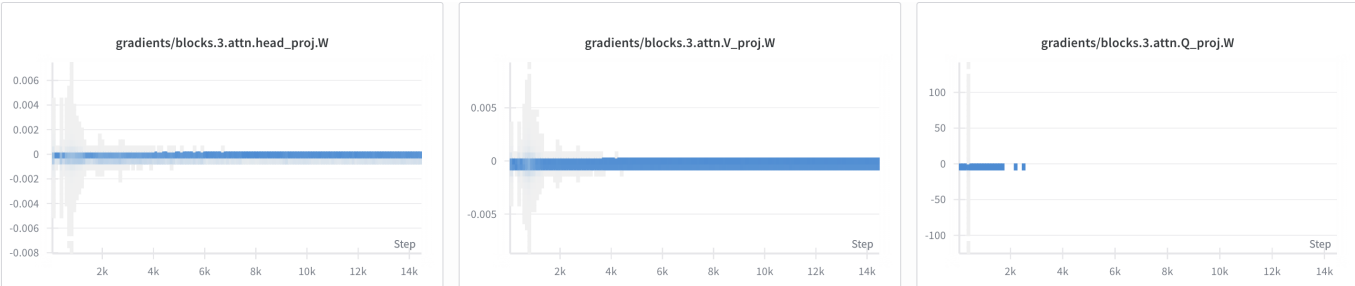
Experiment: LayerNorm, Pre-norm, NoPE, and SwiGLU Ablations

These experiments tested the impact of model architecture choices, including LayerNorm, pre-norm, and SwiGLU. In the first experiment, LayerNorm was removed entirely; the second experiment replaces pre-norm LayerNorm with post-norm. Following experiments removed RoPE position encodings, and SwiGLU with Swish.

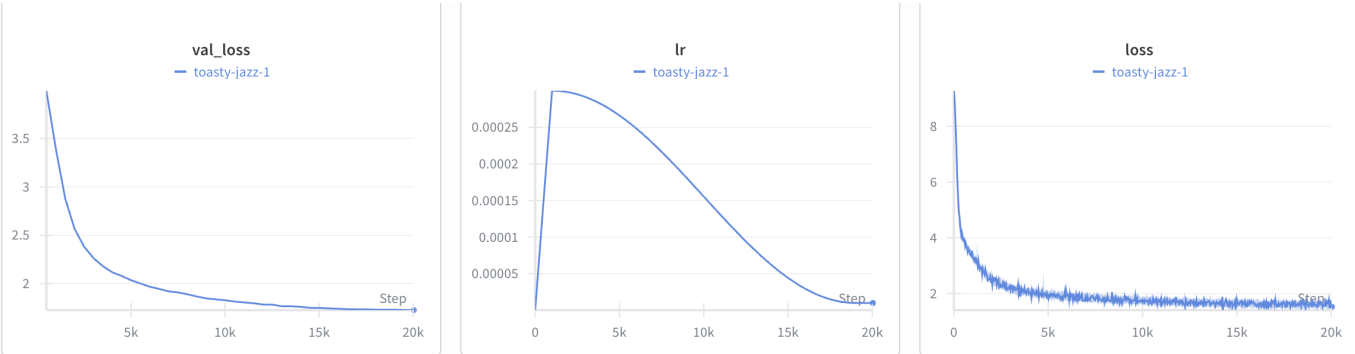
Removing LayerNorm led to significant instability during training (top), which was later mitigated by selecting a lower initial learning rate (bottom):



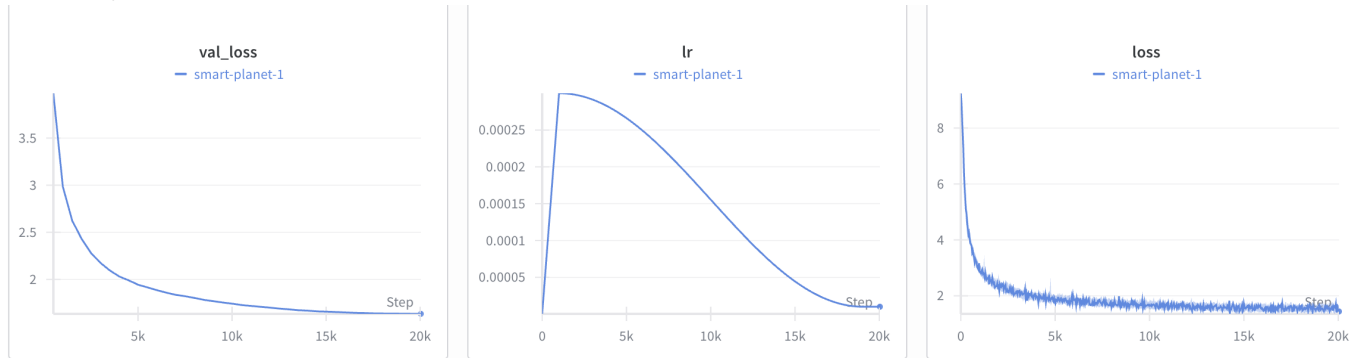
Even with a lower learning rate, validation loss still remained elevated. This is likely due to elevated gradients early in training:



Removing RoPE has a marginal impact, elevating validation loss at 20k steps by around ~0.20:



The story is similar with SwiGLU:



Jan. 19th, 2026

Primary goal for today is to finish out my TSv2 experiments. Before going over results, I'll briefly outline the experimentation setup I've landed on.

Experiment and Compute Setup

I'm using RunPod as my primary GPU provider, for several reasons: it's almost always the cheaper option, solid availability of previous gen GPUs (e.g A-series), and no ingress/egress data pricing.

My compute budget is around $\approx$ \$0.50 per hour. I've experimented with the RTX 6000 Ada and A40; the latter is around $\sim 2\times$ faster than the former while also being $\sim 2\times$ as expensive. I've stuck with the A40 to reduce experiment durations.

All experiments are managed by YAML configs. It's a little cumbersome but is much simpler than remembering how experiment X was configured or relying on logs. Each YAML file is a one-to-many association between a top-level experiment (eg. trying out different batch sizes) and sub-experiments (eg. trying out `batch_size: 32`).

Code is pushed to remote via rsync, with code changes always happening locally. With this setup, writes to remote are idempotent, and there's no need to worry about git credentials or containers. scp is used for remote to local data transfer.

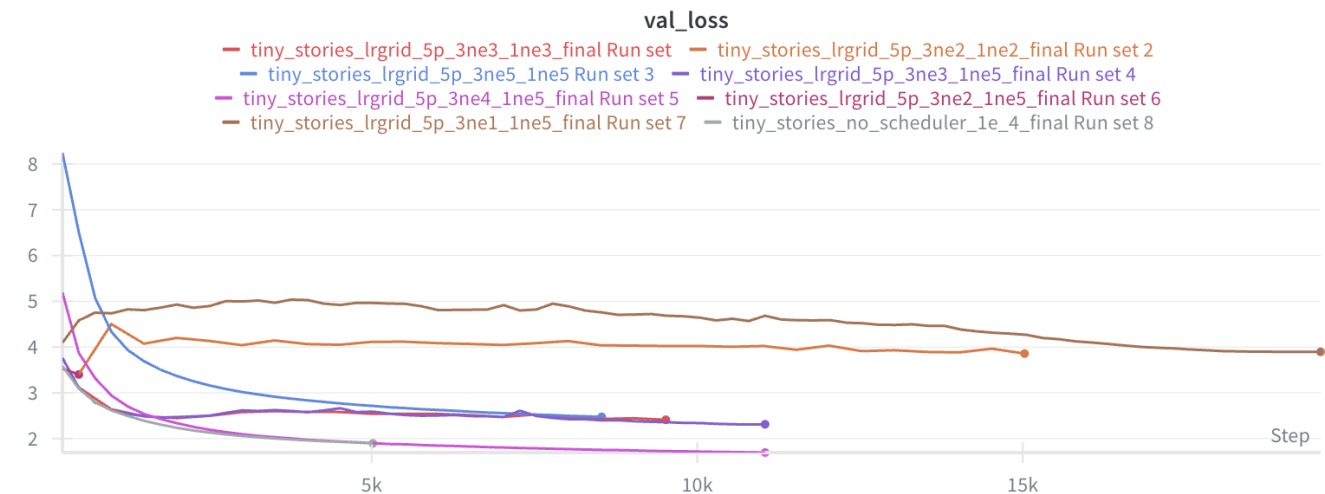
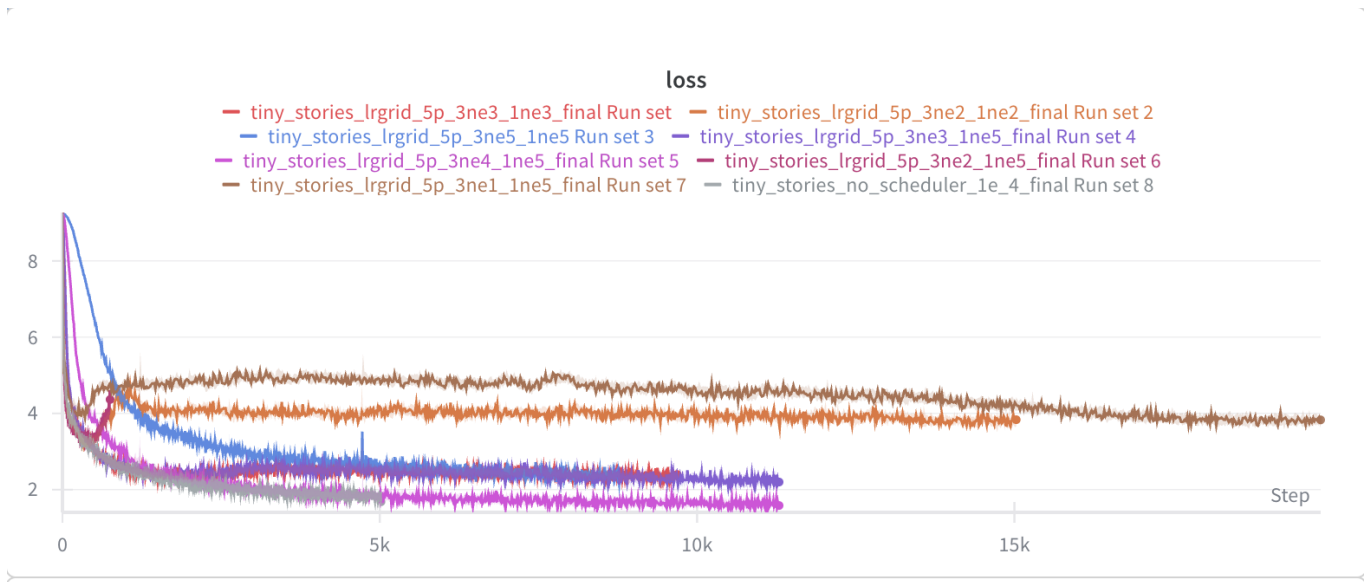
All logging happens over WandB. I was skeptical at first, but it alleviates the need to handle a Tensorboard setup, which I like.

Experiment: LR Sweep

This experiment involved tuning learning rate with all other hyperparams fixed. AdamW is used for all experiments. An cosine annealing LR scheduler is used, outside of [tiny_stories_no_scheduler_1e_4](#).

Name	LR_MIN	LR_MAX	n_warmup	Iterations	Train Loss	Val Loss
tiny_stories_no_scheduler_1e_4	3e-4	3e-4	5%	~ 5000	1.68	1.89
tiny_stories_lrgrid_5p_3ne1_1ne5	3e-1	1e-5	5%	$\sim 20k$	3.82	3.87
tiny_stories_lrgrid_5p_3ne2_1ne5	3e-2	1e-5	5%	750	4.36	3.40
tiny_stories_lrgrid_5p_3ne4_1ne5	3e-4	1e-5	5%	11000	1.58	1.69

Name	LR_MIN	LR_MAX	n_warmup	Iterations	Train Loss	Val Loss
tiny_stories_lrgrid_5p_3ne3_1ne5	3e-5	1e-5	5%	11000	2.19	2.31
tiny_stories_lrgrid_5p_3ne5_1ne5	3e-5	1e-5	5%	8700	2.31	2.47
tiny_stories_lrgrid_5p_3ne2_1ne2	3e-2	1e-2	5%	15000	3.83	3.85
tiny_stories_lrgrid_5p_3ne3_1ne3	3e-3	1e-3	5%	9500	2.39	2.41



These experiments suffer from some step-size bias since some runs terminated early due to node restarts; however, a trend is clear:

- Cosine annealing appears to help, though marginally for smaller models. Notice that the no scheduler and best cosine annealing runs are close in validation loss.
- Large starting max LR's may lead to increasing or flat loss due to too much movement on the loss surface.
- Setting min LR to a smaller LR value helps with learning later in the learning process.

From this, some practical advice:

- Start with a largish LR and slowly taper down until training is stable. A higher LR happens with early exploration and escaping local minima.
- Decaying to a smaller LR after, say 5% of steps, helps with stable learning.
- It may help to keep a higher LR if the model doesn't appear to be learning after a fixed point.

Experiment: Batch Sweep

This is a test in how batch size impacts model performance during training. I attempted several different batch sizes: 1, 64, 128, and 256; I attempted higher batch sizes but subsequently ran out of VRAM. Some of these runs were *not* fully a one-to-one comparison because I did not adjust step size to account for increased token usage per batch; effectively, large runs saw a larger batch size for a higher proportion of the run.

Using a high batch size has proved unstable during training due to OOMs. Turns out I was accumulating gradients during validation and had some other anti-patterns (eg. logging tensor v. item losses)!

Name	BATCH_SIZE	Iterations	Train Loss	Val Loss
tiny_stories_batch_1	1	18k	2.05	2.34
tiny_stories_batch_64	64	11k	1.51	1.62
tiny_stories_batch_128	128	7.5k	1.52	1.62

Large batches fair better than small batches; small batches tend to produce spiky movements over the loss surface due to updating descending along a single example.

Items

- ☒ ~~Finish LR sweep findings~~
- ☒ ~~Finish Batch Sweep~~
 - ☒ ~~256~~
 - ☒ ~~2048~~
- ☐ Ablations
 - ☐ RMSNorm

Jan. 18th, 2026

I'm continuing my training work from yesterday, with the primary goal being to finish out all experiments with TSv2.

GPU Utilization

While running concurrent experiments, I realized *one* experiment was enough to saturate the GPU:

```
root@f4eb97ed8cec:/workspace/tahoe# nvidia-smi
Mon Jan 19 02:00:34 2026

+-----+
| NVIDIA-SMI 570.195.03                Driver Version: 570.195.03      CUDA Version: 12.8     |
+-----+-----+
| GPU  Name           Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|                                           | MIG M.         |
+-----+-----+
|  0  NVIDIA A40          On        | 00000000:05:00.0 Off |           0         |
|  0%   71C    P0         272W / 300W | 6919MiB / 46068MiB |   97%      Default  |
|                                           | N/A             |
+-----+-----+

+-----+
| Processes: |
| GPU   GI    CI          PID    Type   Process name                        GPU Memory |
|   ID   ID     ID              |                 | Usage     |
+-----+-----+
|  0    N/A   N/A         2335     C   ...space/tahoe/.venv/bin/python3   6912MiB |
+-----+-----+
```

Notice that the GPU is running at 97% utilization. In this state, launching another experiment is fairly counterproductive; namely, each process experiences a reduction in FLOPS due to context switching between work. I'll keep this in mind in the future.

LR Sweep Results

More curves to come tomorrow!

Items

- ☒ Refactor codebase to support concurrent experiments
- ☒ No scheduler run
- ☒ Perform hyperparameter sweep
 - ☒ `tiny_stories_lrgrid_5p_3ne1_1ne5`
 - ☒ `tiny_stories_lrgrid_5p_3ne2_1ne5`
 - ☒ `tiny_stories_lrgrid_5p_3ne3_1ne5`
 - ☒ `tiny_stories_lrgrid_5p_3ne4_1ne5`
 - ☒ `tiny_stories_lrgrid_5p_3ne5_1ne5`
 - ☒ `tiny_stories_lrgrid_5p_3ne2_1ne2`
 - ☒ `tiny_stories_lrgrid_5p_3ne3_1ne3`
- ☐ Perform batch size sweep
 - ☐ 64, 128, 512, 1024, 4096

Jan. 17th, 2026

I have a complete model architecture, tokenized data, and a training loop. The next step is to train a model and understand the processes around training.

Two datasets to cover: TinyStoriesV2 (TSv2) and sampled OpenWebText (OWT). I'll focus most of my time on TSv2, at least for today, because it's less resource intensive. There are a number of hyperparameters to tune, particularly model architecture (eg. d_{model}), LR, and batch size. Concretely:

	Parameter	Category	GPT-2	cs336 (TSv2)
1	vocab_size	data	50,257	10,000
2	batch_size	data	512	Tentative
3	context_length	data / model	1024	256+
4	num_layers	model	12-48	4
5	d_model	model	768-1600	512
6	num_heads	model	12-25 ^a	16
7	d_ff	model	3072-6400	1344
8	theta	model	N/A	10000
9	lr_max	LR / hyperparameter	2.5e-4	Tentative
10	lr_min	LR / hyperparameter	N/A	Tentative
11	t_warmup	LR / hyperparameter	N/A	Tentative
12	t_cos	LR / hyperparameter	N/A	Tentative
13	betas	AdamW / hyperparameter	0.9, 0.999	Tentative
14	weight_decay	AdamW / hyperparameter	0.01	Tentative
15	max_grad	hyperparameter	1.0	Tentative

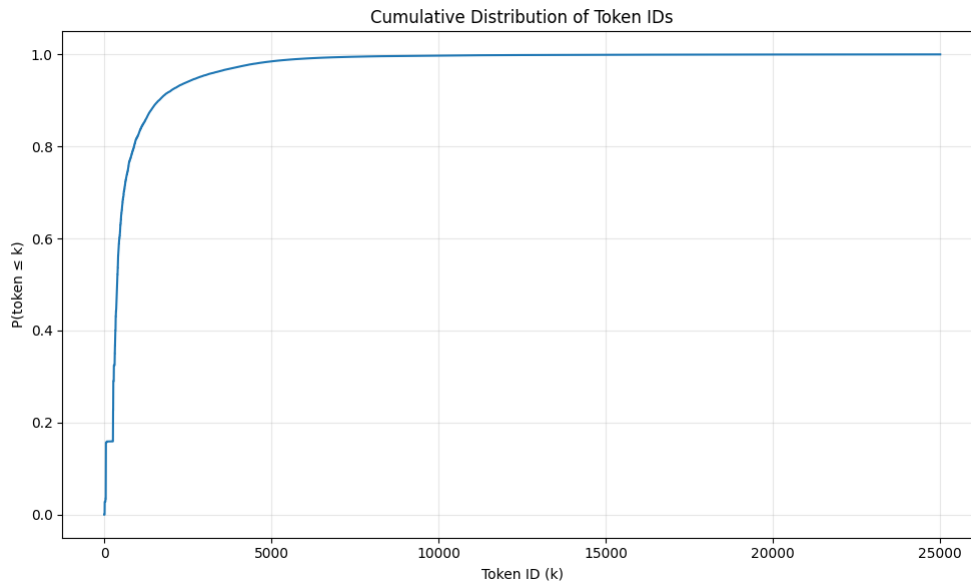
a - chosen to be a multiple of 64.

Data

The simplest place to start is with **data** and some essential questions:

- How large of a vocab should we use? The is the long tail of tokens? What should be the truncation value?
 - **A.** For TSv2, 10k is a solid truncation point, covering ~99.9% of training tokens.
- How many tokens should a given training run use?
 - **A.** $\geq 20 \cdot \text{model size}$, according to Chinchilla scaling laws.

Vocab size impacts embedding layer and LM layer weights, and for a small model, these values will contribute significantly to model size. For TSv2, the cumulative distribution of tokens looks like:



Around 10k tokens, the CDF effective converges to 1.0, with a density loss of <0.1%:

```
# yielded by `tokens.py`
> 0: 0.0; density_loss=1.0; token_loss=536813210
> 1000: 0.8227953201822288; density_loss=0.17720467981777122; token_loss=95125813
> 2000: 0.9216566745814619; density_loss=0.07834332541853806; token_loss=42055732
> 3000: 0.9545258079621401; density_loss=0.04547419203785985; token_loss=24411147
> 4000: 0.9729588621710706; density_loss=0.027041137828929362; token_loss=14516040
> 5000: 0.9848745879409339; density_loss=0.015125412059066146; token_loss=8119521
> 6000: 0.9909489298894116; density_loss=0.009051070110588366; token_loss=4858734
> 7000: 0.9940057194196097; density_loss=0.00599428058039031; token_loss=3217809
> 8000: 0.9957168714234883; density_loss=0.004283128576511697; token_loss=2299240
> 9000: 0.9967888197088145; density_loss=0.0032111802911855003; token_loss=1723804
> 10000: 0.9975380821943632; density_loss=0.0024619178056367597; token_loss=1321590
```

Truncation is probably safe around that point. I can either compact the vocab or random sample. I'll probably just compact the vocab, training, and validation data. Both approaches prevent biasing towards a particular token. After further investigation, I realized I made a mistake and 10k is the *expected vocab size*.

The next point is on training set size provided model capacity, which is turn is driven by GPU limits. Chinchilla's guidance is 20 tokens per parameter. Llama overtrains by a factor of 10x? It seems like the general guidance is as much tokens as possible. The regime you want to avoid is large model + minimal tokens.

Working backwards, we have 538,511,524 training tokens with TSv2; assuming Chinchilla scaling laws, this yields a model size of $\leq 538,511,524/20 \approx \boxed{25M}$ parameters. This is a rough limit, agnostic of GPU resources.

Small update. My tokenization code was wrong. `<|endoftext|>` tokens were being filtered out :(My decoding code was also wrong. I should've sanity checked the outputs for these files.

Model

Based on the analysis above, the model should require $\leq 25M$ parameters. I'll rely on the parameters provided with cs336.

I was able to train the model to a reasonable state on RunPod. Next, I'll handle grid searching for hyperparams.

Compute

I don't need a ton of compute for these tests. I'm considering the following test setup:

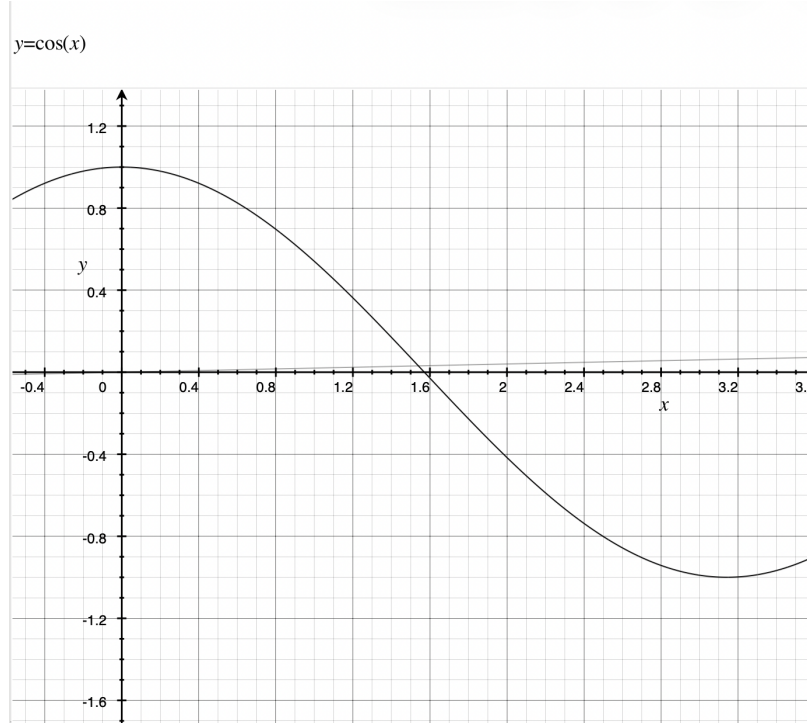
- RTX 4000 Ada (20GB vRAM) on RunPod.
- Nvidia A10 on Lambda Labs

Training Dynamics

This section is about optimizers, particularly hyperparameters. The first area of focus is *learning rate*, which controls the step size in gradient descent. The model is using a cosine scheduler that's parameterized by:

- t : the current iteration.
- $\alpha_{\max}$: the max learning rate.
- $\alpha_{\min}$: the min learning rate.
- T_w : the warmup learning rate.
- T_c : the cosine learning rate.

The cosine scheduler starts by warming up (ie. increasing) the learning rate $\frac{1}{T_w} \alpha_{\max}, \dots, \alpha_{\max}$; linear growth. After warm-up, the learning rate is decayed via a cosine-interpolation until it hits $\alpha_{\min}$ at T_c :



Experiment 1: Basic Fitting

This run leveraged an RTX 4000 Ada on RunPod.

The parameters for this run are under `./configs/tiny-stories-1.yaml`. Training loop was run for 5000 iterations, yielding $32 * 256 * 5000 \approx 40M$ seen tokens.

Training loss reduced to ≤ 2.0 :



Validation loss was not monitored, unfortunately. After 5000 iterations, the model was able to generate realistic sounding English sentences. The training set, however, did not include any special tokens, including `<|endof text|>` tokens, leading to meandering responses. I need t

Items

- ☒ ~~Truncate TSv2 to 10k tokens:~~
 - ☒ ~~Investigation estimated token coverage:~~
 - ☒ ~~Retrain vocab:~~
 - ☒ ~~Retrain train set on 10k reduction:~~
 - ☒ ~~Retrain val set on 10k reduction:~~
- ☒ ~~Retrain TSv2 and OWT datasets:~~
 - ☒ ~~Debug missing special tokens:~~
 - ☒ ~~Retrain OWT:~~
 - ☒ ~~Retrain TSv2:~~
- ☒ ~~o11y~~
 - ☒ ~~Enable wandb logging for validation loss:~~
- ☐ Training Validation
 - ☐ Validation loss drop over 1000 iterations.
- ☐ Parallel grid search / sweep for hyperparams